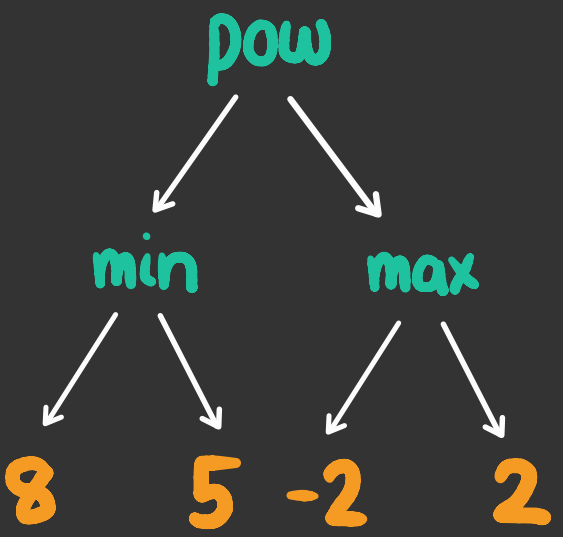
recursion

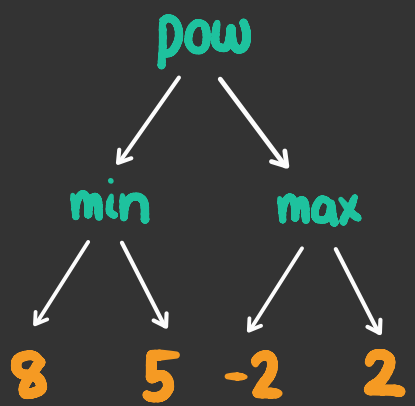
CSCI 134

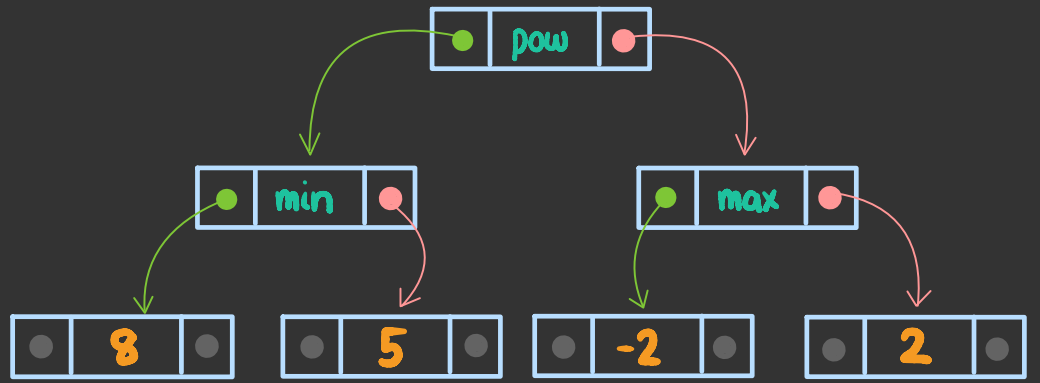
the first kind of tree we encountered

was an expression tree



we now have a way to represent trees in python



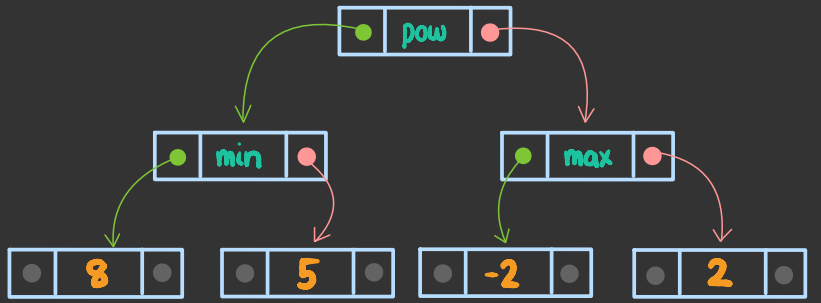


we now have a way to represent trees in python

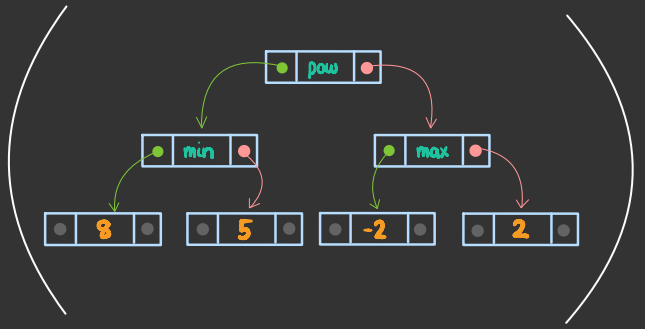
```python
def tree_node(l, lbl, r):
    return [l, lbl, r]

def leaf(lbl):
    return tree_node(None, lbl, None)

def example_expression_tree():
    eight = leaf(8)
    five = leaf(5)
    negative_two = leaf(-2)
    two = leaf(2)
    max_node = tree_node(negative_two, max, two)
    min_node = tree_node(eight, min, five)
    root = tree_node(min_node, pow, max_node)
    return root
```

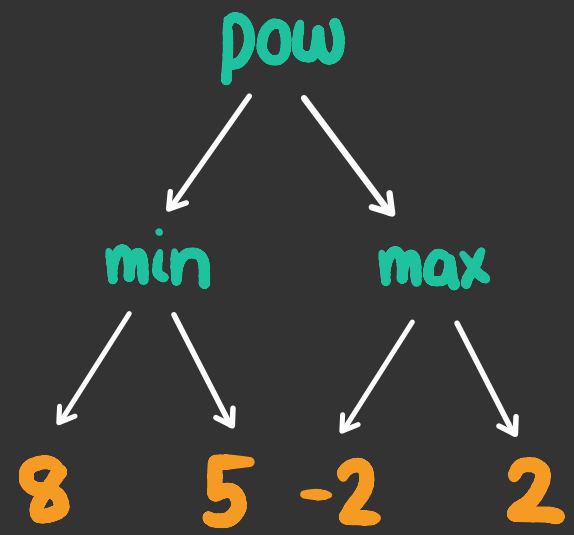


Can we also write code to evaluate an expression tree?

evaluate 

= 25

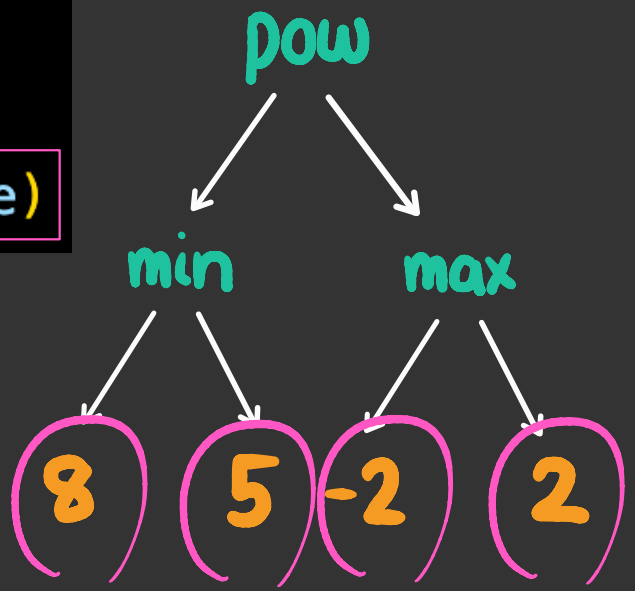
recall that we evaluate an expression tree "bottom-up"



so leaf nodes are easy to evaluate

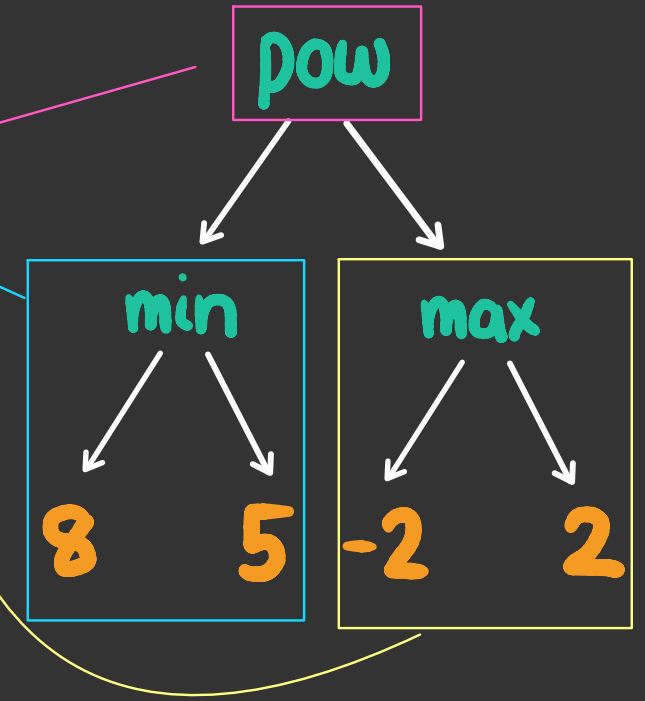
```
def evaluate(etree):
    if is_leaf(etree):
        return label(etree)
```

the value of a leaf node is just its label



the value of a non-leaf is its label (a function) applied to the values of its children

```
def evaluate(etree):
    if is_leaf(etree):
        return label(etree)
    else:
        func = label(etree)
        left_value = evaluate(left(etree))
        right_value = evaluate(right(etree))
        return func(left_value, right_value)
```

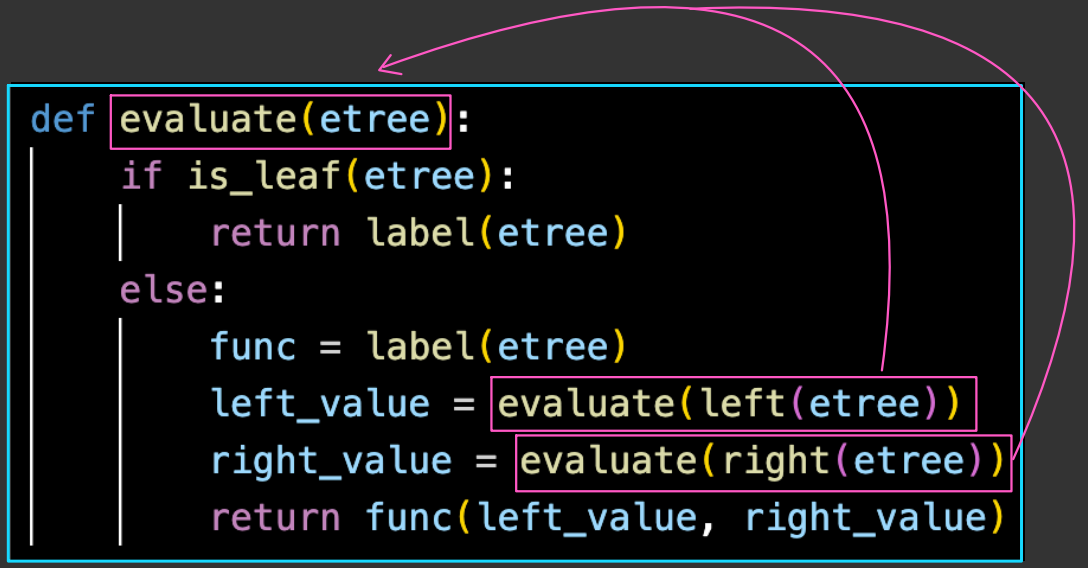


this is an example of a recursive function

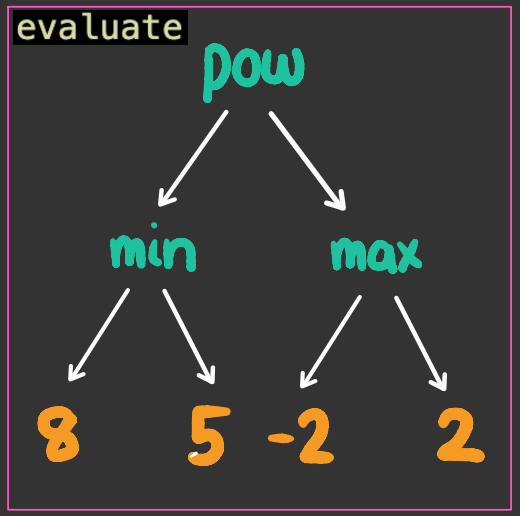
```
def evaluate(etree):
    if is_leaf(etree):
        return label(etree)
    else:
        func = label(etree)
        left_value = evaluate(left(etree))
        right_value = evaluate(right(etree))
        return func(left_value, right_value)
```

a recursive function

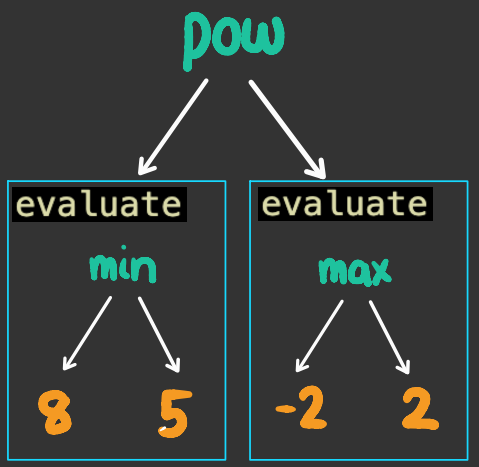
is a function that calls itself



recursion is appropriate when we can solve a task by breaking it into smaller versions of the same task

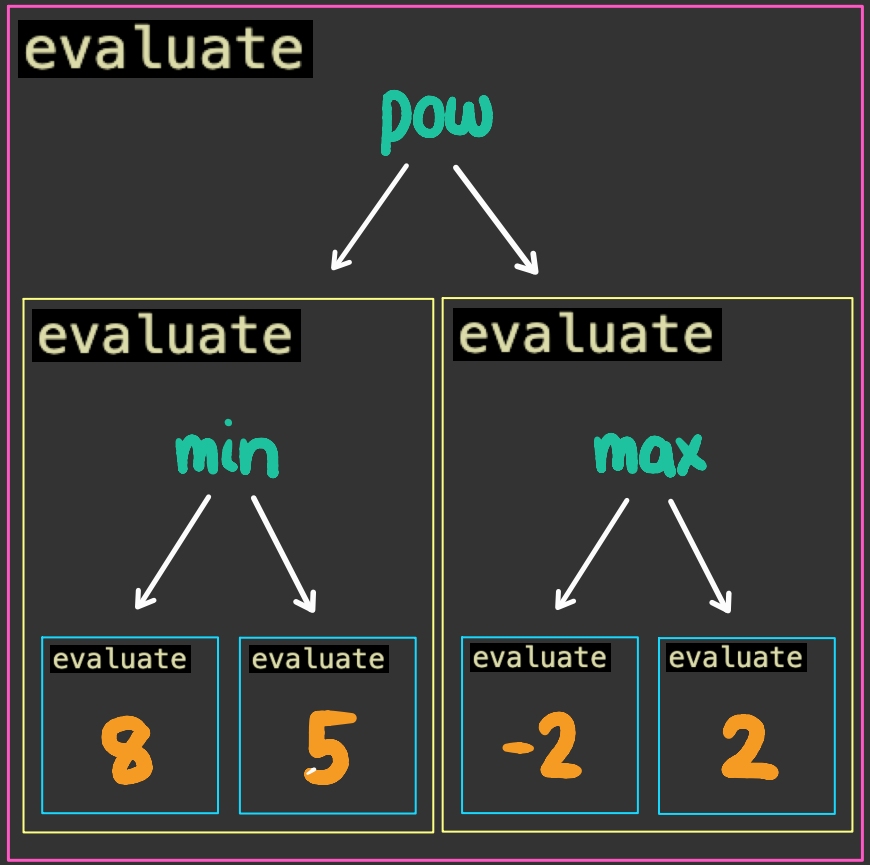


we can compute the value of this tree



if we can compute the values of these trees

by breaking up the task into smaller and smaller tasks, eventually the tasks become small enough to solve directly



these "small enough" tasks are called base cases

```
def evaluate(etree):
    if is_leaf(etree):
        return label(etree)
    else:
        func = label(etree)
        left_value = evaluate(left(etree))
        right_value = evaluate(right(etree))
        return func(left_value, right_value)
```